

Auth, JWT & npm Versions

login, register, reset password, how JWT tokens work, and what ~ ^ mean in package.json

Today ties together the pieces that let a user have an account: register, login, reset password, and the token that keeps them logged in. Then we look inside a JWT token to see how it actually works, and finish with a small but important thing you see every day in package.json, the ~ and ^ symbols in front of version numbers. Every code example here was written and run before it reached this page.

1 The three auth endpoints, in one picture

An account needs three doors. They all revolve around one rule: the password is never stored as typed.

Endpoint	What it does
register	make a new account, hash the password, hand back a token
login	check the password, hand back a fresh token
reset password	replace the stored hash with a new one

2 Register

Register takes a name, email and password. It checks the email is not already used, hashes the password, saves the user, and returns a token so the person is logged in immediately after signing up.

register

```
function register(name, email, password) {
  if (users.find(u => u.email === email))
    throw new Error('Email already registered');

  const hash = bcrypt.hashSync(password, 10); // one-way, never store plain
  const user = { id: nextId(), name, email, password: hash };
  users.push(user);

  const token = jwt.sign({ id: user.id }, SECRET, { expiresIn: '7d' });
  return { id: user.id, name, email, token };
}
```

```
register -> { id: 1, name: 'Himanshu', email: 'h@x.com', token: 'eyJhbGciOiJIU...' }
```

3 Login

Login finds the user by email, then checks the password against the stored hash. If either fails, it returns the same "Invalid credentials" error, never "wrong password" separately, so an attacker cannot tell which part was wrong.

login

```
function login(email, password) {
  const user = users.find(u => u.email === email);
  if (!user || !bcrypt.compareSync(password, user.password))
    throw new Error('Invalid credentials');

  const token = jwt.sign({ id: user.id }, SECRET, { expiresIn: '7d' });
  return { token };
}
```

```
login correct password -> token issued
login WRONG password -> rejected: Invalid credentials
```

Why the same error for both cases. If a wrong email said "no such user" and a wrong password said "wrong password", an attacker could learn which emails are registered. One vague error for both closes that door.

4 Reset password

Reset finds the user and overwrites the stored hash with a hash of the new password. After this, the old password stops working, exactly what you want.

reset password

```
function resetPassword(email, newPassword) {
  const user = users.find(u => u.email === email);
  if (!user) throw new Error('User not found');

  user.password = bcrypt.hashSync(newPassword, 10); // hash the new one too
  return { message: 'Password updated' };
}
```

after reset:
old password -> rejected: Invalid credentials
new password -> works

5 Why the password is stored hashed

Run the demo and look at what actually sits in the database:

stored password: \$2b\$10\$KR0tMiO/16JH51Bipc3EPuWnSvA9q6QxRx51R59WI3YuCCvNJ8Smu

That scramble is a one-way hash. You can turn a password into it, but you cannot turn it back into the password. At login, `bcrypt.compareSync` hashes the fresh attempt and compares the two scrambles. So even you, looking at the database, cannot read anyone's password, and a leaked database does not hand over real passwords. This is the one-way encryption idea from yesterday, now in code.

6 What a JWT token actually is

The token looks like random text, but it is not. A JWT is three parts joined by dots: `header.payload.signature`. Split one open and read it:

the three parts, decoded

```
HEADER    { alg: 'HS256', typ: 'JWT' }
PAYLOAD   { id: 5, role: 'student', iat: ..., exp: ... }
SIGNATURE QWWHS6sXqBYOHMTtPQm5JJ0imJ95FfNUmsgdfCIQGGg
```

- **Header** says which algorithm signed it.
- **Payload** is the actual data, here the user id. This is the part your middleware reads to know who is calling.
- **Signature** is a fingerprint made from the header, payload and your secret key.

The payload is NOT secret. Anyone holding the token can read the payload, it is only encoded, not encrypted. So never put a password or anything private in a JWT. Put an id, and look the rest up in your database.

7 Why a tampered token is caught

If the payload is readable, why can't someone change their id to 1 and become the admin? Because of the signature. Change even one character and the signature no longer matches, and `jwt.verify` throws.

verify a real token -> user: Himanshu
verify a changed token -> rejected: invalid signature

This is the whole trick of JWT. The token is readable but not forgeable. Only your server knows the secret key, so only your server can produce a valid signature. An attacker can read a token, but the moment they change it, it stops verifying. That is how one line, `jwt.verify(token, SECRET)`, protects every route.

8 npm versions: what ~ ^ and no symbol mean

Open any package.json and you see lines like `"express": "^4.17.21"`. That symbol controls how far npm is allowed to update the package. First, the version number itself:

MAJOR . MINOR . PATCH, for example 4.17.21

PATCH (21) goes up for a bug fix, nothing else changes. MINOR (17) goes up for a new feature that does not break old code. MAJOR (4) goes up when something breaks and your code may need changing.

Now the symbols. We tested these against real versions:

In package.json	Meaning	Updates allowed from 4.17.21
4.17.21	exact, no symbol	only 4.17.21, nothing else
~4.17.21	tilde: patch updates only	up to 4.17.22, 4.17.99, but never 4.18.0
^4.17.21	caret: minor and patch	up to 4.18.0, 4.99.9, but never 5.0.0

verified with npm's own semver tool

```
4.17.21 (no symbol) allows: 4.17.21
~4.17.21 (tilde)    allows: 4.17.21, 4.17.22      (patch only)
^4.17.21 (caret)   allows: 4.17.21, 4.17.22, 4.18.0, 4.99.9 (never 5.0.0)
```

The simple way to remember it. ^ caret is the npm default, it lets in new features and fixes but never a breaking major change. ~ tilde is stricter, only bug-fix patches. No symbol is locked, exactly this version and no update at all. The rule they share: none of them ever cross a major version on their own, because a major bump can break your code.

9 Your task

Read and cover all of this: the login, register and reset password endpoints, JWT tokens, and the npm version symbols.

Prove the auth flow to yourself. Register a user, log in, and confirm a wrong password is rejected. Look at the stored password and see that it is a hash, not the real password.

Open a real JWT. Take a token, split it on the dots, and decode the middle part. See your own user id sitting in the payload. Then change one character and watch verify reject it.

Look at your package.json. Find the ^ and ~ symbols. For one package, say out loud which updates it will accept and which it will refuse.

Write each topic in your own words in your notes. Understanding these is understanding how every login you have ever used actually works.